

 DATA ENGINEERING

Data Engineering For Fine-Tuning

The Complete Guide

Your data quality IS your model quality.
No architecture can fix bad training data.

Sources

Synthetic Gen

Cleaning

Validation

Quality Eval



AI Coach John

FOLLOW



PROITBRIDGE
Smart For Smarter Futures

 Repost

80% of Fine-Tuning Is Data Engineering

While many focus on model architectures and hyperparameters, the actual determinant of success is your data. Poor data pipelines will ruin the most advanced LLMs.

⚠️ WHY PROJECTS FAIL

- > Data too small → model can't generalize
- > Data inconsistent → model learns noise, not patterns
- > Data duplicated → model overfits, metrics lie
- > Data in wrong format → model ignores structure
- > Data has wrong distribution → fails on real inputs

TIME DISTRIBUTION IN REAL PROJECTS



💡 The industry maxim: **"80% of ML is data work"** - fine-tuning is no exception. Clean, well-structured data beats complex architectures every time.



Data Sources - Where to Find Quality Data

Before cleaning or formatting, you must source high-fidelity data. Start with data closest to the end-user's task, then augment with synthetic or public data as a last resort.

1. Internal production data

Best signal. Real user queries + expert responses. Requires quality filter $\geq 4/5$ rating.

2. Expert annotation

Domain experts write examples from scratch. Most expensive, highest quality. Use Argilla or LabelStudio.

3. Public datasets

Alpaca (52K), Dolly (15K), OpenHermes (1M), ShareGPT (90K). Check license before use.

4. Synthetic generation (AI-assisted)

Claude / GPT-4 generates examples from seed data. Scales cheaply - quality needs human QA.

5. Web scraping (structured)

Stack Overflow, GitHub issues, docs. Heavy filtering required. Use only open-license content.

QUALITY TIER RANK

Production logs > Expert annotation > Synthetic (GPT-4) > Public datasets > Web scraped



AI Coach John

FOLLOW



Repost

Synthetic Data Generation - Scale Without Humans

i WHEN TO USE SYNTHETIC DATA

You have fewer than 500 real examples. Production data exists but can't be labeled at scale. Domain is niche with few public datasets.

```
from anthropic import Anthropic
client = Anthropic()

def gen_synthetic(seed: dict, n: int = 10) → list[dict]:
    sys_prompt = f"Generate {n} diverse pairs (JSON)."
    msg = client.messages.create(
        model="claude-opus-4-8", max_tokens=4000,
        system=sys_prompt,
        messages=[{"role": "user",
                    "content": f"Seed: {json.dumps(seed)}"}]
    )
    return json.loads(msg.content[0].text)

all_examples = []
for seed in [{"instruction": "...", "response": "..."}]:
    all_examples.extend(gen_synthetic(seed, n=20))
```

★ **Self-instruct:** Generate instructions from documents, then ask LLM to answer them. Scales domain data from near-zero.



Instruction Design - Teaching the Right Behavior

✖ BAD INSTRUCTIONS

```
# Vague, inconsistent
{"instruction": "help me",
 "response": "Sure!"}
{"instruction": "AI explain",
 "response": "AI is tech"}
```

✔ GOOD INSTRUCTIONS

```
{"instruction": "Explain retrieval-
augmented generation to a software
eng. Audience: knows Python, new to
LLMs.", "response": "RAG is a pattern
that separates knowledge from the
model itself..."}
```

! ISSUES

- > Too short / vague
- > No context in instruction
- > Inconsistent format
- > Missing audience

☰ QUALITY RULES

- > Specific, scoped task
- > Audience stated
- > Consistent format
- > Response ≥ 3 sentences



Diversity checklist: vary difficulty, question type (HOW/WHY/WHAT/COMPARE), audience, format, length. Aim for ≥ 5 dimensions of variation across dataset.



Preference Data - Teaching Chosen vs Rejected

📌 WHAT PREFERENCE DATA LOOKS LIKE

Three fields per example: **prompt** (what the user said), **chosen** (the preferred response), **rejected** (the worse response). Used for DPO alignment training.

```
# Human-annotated preference data format
preference_example = {
    "prompt": "What is quantum computing?",
    "chosen": "" "Quantum computing uses quantum bits (qubits)
that can exist in superposition, enabling certain
calculations exponentially faster than classical bits.
Key applications: cryptography, drug discovery, optimization.
Current limitation: hardware requires near-absolute-zero temperatures.""",
    "rejected": "Quantum computers are really complex and very
powerful. They use quantum physics. It's complicated."
}

# Generate synthetic preference data with LLM
def create_preference_pair(prompt: str) → dict:
    chosen = strong_model.generate(prompt,
        system="Expert, detailed, structured answer.")
    rejected = weak_model.generate(prompt,
        system="Quick, brief response.")
    return {"prompt": prompt,
        "chosen": chosen, "rejected": rejected}
```

★ **Tip:** You need ~1K–5K preference pairs for DPO. Synthetic generation (strong model = chosen, weak = rejected) works well for format/style alignment.



Conversation Formatting - The Chat Template Matters

6 MULTI-TURN FORMAT (SHAREGPT / CHATML)

```
# ShareGPT format (for multi-turn conversations)
multi_turn = {
    "conversations": [
        {"from": "system",
         "value": "You are a helpful AI tutor."},
        {"from": "human",
         "value": "What is fine-tuning?"},
        {"from": "gpt",
         "value": "Fine-tuning adapts a pre-trained model ..."},
        {"from": "human",
         "value": "How is it different from RAG?"},
        {"from": "gpt",
         "value": "RAG retrieves external knowledge at inference ..."}
    ]
}

# HuggingFace chat template application
def apply_chat_template(example, tokenizer):
    messages = example["conversations"]
    formatted = tokenizer.apply_chat_template(
        messages,
        tokenize=False,
        add_generation_prompt=True
    )
    return {"text": formatted}
```

⚠ Critical: Apply the same chat template at training and inference. A mismatch here causes silent quality degradation - the model sees different input structure than it was trained on.



Cleaning Pipeline - 5 Stages That Cannot Be Skipped

Raw data is never ready for fine-tuning. A rigorous cleaning pipeline ensures the model focuses on learning the task rather than memorizing anomalies or noise.

- 1 **PII scrubbing** - Remove names, emails, phone numbers, SSN using regex + NER model before any storage or training.
- 2 **Length filtering** - Drop instructions < 10 chars, responses < 50 chars. They add noise without signal.
- 3 **Quality scoring** - LLM-judge rates each example 1-5. Drop anything below 3.5. Use Claude Haiku for cost efficiency.
- 4 **Toxicity check** - Run Perspective API or a fine-tuned classifier. Remove harmful content before it goes into training.
- 5 **Format validation** - Parse every example through the chat template. If it fails to parse, the model can't learn from it.

```
def cleaning_pipeline(examples: list) → list:
    examples = [scrub_pii(e) for e in examples]
    examples = [e for e in examples
                 if len(e["response"]) > 50]
    examples = quality_score_batch(examples)
    examples = [e for e in examples if e["score"] ≥ 3.5]
    return [e for e in examples if not is_toxic(e)]
```



Deduplication - Duplicates Destroy Generalization

⚠ WHY DUPLICATES ARE DANGEROUS

Exact or near-duplicate examples: inflate your dataset size metric, cause the model to overfit those specific patterns, artificially improve eval scores (eval set may contain the same duplicate), and prevent generalization to unseen inputs.

= EXACT DEDUP

- MD5 hash of the instruction text
- Catches copy-paste duplicates
- O(n) time, zero cost

↗ SEMANTIC DEDUP

- Embed → cosine similarity
- Threshold: 0.92–0.95
- Catches paraphrase dups

```
import hashlib
from sentence_transformers import SentenceTransformer
from sklearn.metrics.pairwise import cosine_similarity

def deduplicate(examples):
    seen, unique = set(), []
    for ex in examples: # Exact dedup
        h = hashlib.md5(ex["instruction"].encode()).hexdigest()
        if h not in seen: seen.add(h); unique.append(ex)

    # Semantic dedup
    model = SentenceTransformer("all-MiniLM-L6-v2")
    embs = model.encode([e["instruction"] for e in unique])
    sim = cosine_similarity(embs)
    to_drop = {j for i in range(len(sim))
                for j in range(i+1, len(sim)) if sim[i,j]>0.95}
    return [e for i,e in enumerate(unique) if i not in to_drop]
```



✓ STEP 8

Schema Validation - Every Example Must Parse

i WHY VALIDATE SCHEMA

Training will silently fail or degrade if even 1% of examples have wrong keys, missing fields, or encoding errors. Validate everything before the training loop sees it.

```
from pydantic import BaseModel, validator

class TrainingExample(BaseModel):
    instruction: str
    response: str

    @validator("instruction")
    def valid_inst(cls, v):
        assert len(v.strip()) ≥ 10; return v.strip()

    @validator("response")
    def valid_resp(cls, v):
        assert len(v.strip()) ≥ 50; return v.strip()

def validate_dataset(raw: list[dict]):
    valid, invalid = [], []
    for i, ex in enumerate(raw):
        try: valid.append(TrainingExample(**ex).dict())
        except Exception as e: invalid.append({"idx": i})
    return valid, invalid

valid_data, errors = validate_dataset(raw_data)
```

🛡️ Run validation as a CI/CD step before every training job. Catch schema issues in seconds, not after a 6-hour training run.



AI Coach John

FOLLOW



Repost

Train / Val / Test Split - Do It Right or Cheat Yourself

RECOMMENDED SPLITS

Training		80%
Validation		10%
Test (held-out)		10%

⚠ THE TEST SET RULES

- > Touch it **ONLY** once - after training is complete
- > Never use it to tune hyperparameters (use val set)
- > Never let near-duplicates span train/test

```
from sklearn.model_selection import train_test_split
import random

def stratified_split(
    dataset: list,
    train_ratio=0.8,
    val_ratio=0.1) → tuple:
    # Shuffle with fixed seed for reproducibility
    random.seed(42)
    random.shuffle(dataset)
    n = len(dataset)
    train_end = int(n * train_ratio)
    val_end = int(n * (train_ratio + val_ratio))
    return (dataset[:train_end],          # train
            dataset[train_end:val_end],   # validation
            dataset[val_end:])            # test (hold-out))
```



Data Quality Evaluation - Before Training Begins

Q 5 QUALITY CHECKS BEFORE TRAINING

- **Distribution check** - Are topics evenly distributed? Sample and visualize instruction topics.
- **Length distribution** - Plot instruction and response lengths. Outliers hurt batching.
- **Vocabulary coverage** - Does training data cover your domain terms? Check top-100 domain words appear.
- **LLM-judge random sample** - Score 100 random examples. Mean score must exceed 3.8/5.0.
- **Human spot-check** - A domain expert reviews 50 random examples. Non-negotiable gate.

```
from anthropic import Anthropic
import random

def evaluate_data_quality(dataset, sample_n=100):
    sample = random.sample(dataset, min(sample_n, len(dataset)))
    client = Anthropic()
    scores = []
    for ex in sample:
        resp = client.messages.create(
            model="claude-haiku-4-5-20251001",
            max_tokens=100,
            messages=[{"role": "user", "content":
                f"""Rate this training example 1-5:
Instruction: {ex['instruction']}
Response: {ex['response']}"""
            }]
```



Data Engineering Mistakes That Kill Models

Even with a robust pipeline, simple oversights can destroy your model's performance. Avoid these common anti-patterns to ensure generalization and safety.

- 1 **Train/test leakage** - Near-duplicates split across train and test. Metrics look great; real performance is terrible.
- 2 **Format inconsistency** - Mixing Alpaca + ShareGPT + raw text in one run. Model can't learn a consistent output pattern.
- 3 **Unvalidated synthetic data** - LLM-generated data used without human review. Confident hallucinations baked into the model.
- 4 **PII in training data** - Model memorizes and regurgitates PII at inference. Legal liability + GDPR violation.
- 5 **Imbalanced distribution** - 90% of data is one topic. Model excels at that topic, forgets everything else (catastrophic forgetting).
- 6 **Too few examples** - Under 200 examples? Model won't generalize. You'll see overfitting from epoch 1.
- 7 **Skipping the eval step** - Never checking data quality before training. You won't know the data failed until training does.



Data Engineering Pipeline Cheatsheet

STEP	WHAT	TOOL / GATE
1. Source	Collect from production, experts, public	Argilla, HF Datasets
2. Generate	Synthetic from seeds if <500 examples	Claude Opus / GPT-4
3. Design	Diverse instructions, varied difficulty	Manual checklist
4. Preferences	Chosen/rejected for DPO alignment	LLM judge pair gen
5. Format	Apply chat template consistently	Tokenizer template
6. Clean	PII, length, quality, toxicity	5-stage pipeline
7. Dedup	Exact + semantic deduplication	SentenceTransformers
8. Validate	Schema check every example	Pydantic validators
9. Split	80/10/10 train/val/test (no leakage)	Fixed-seed shuffle
10. Eval	LLM-judge score + human spot-check	Gate: mean $\geq 3.8/5$

💡 The Fundamental Rule

1,000 high-quality examples beat 100,000 dirty ones. Every time. Invest in data quality before architecture complexity.



AI Coach John

FOLLOW



↻ Repost